Asynchronous JavaScript: Promises, Fetch & Async/Await

Continuation from Part 1: Callbacks & XMLHttpRequest

Introduction to Promises

A Promise is an object that represents the eventual completion (or failure) of an asynchronous operation and its resulting value. Think of it as a receipt for work that hasn't finished yet—you can't change the receipt itself, but you can ask to be notified when the work is done or if it fails.

Promises were introduced to address the callback hell problem we encountered in Part 1. They provide a cleaner, more manageable way to handle asynchronous operations.

Promise States

A Promise exists in one of three states:

- **Pending:** Initial state, the operation is still in progress
- **Fulfilled:** The operation completed successfully, and a value is available
- **Rejected:** The operation failed, and an error reason is available

Once a Promise is fulfilled or rejected, it becomes immutable—its state and value cannot change.

Creating a Promise

Let's refactor our XMLHttpRequest example from Part 1 to return a Promise instead of using callbacks:

```
function getPromise(url, delay) {
    return new Promise((resolve, reject) => {
        setTimeout(() => {
            const xhr = new XMLHttpRequest();

            xhr.open("GET", url);

            xhr.onload = function() {
```


Powered by SPRK Technologies

```
                if (xhr.status >= 200 && xhr.status < 300) {
                    resolve(JSON.parse(xhr.responseText));
                } else {
                    reject(new Error("Request Failed"));
                }
            }

            xhr.onerror = function() {
                reject(new Error("Network error"));
            }

            xhr.send();
        }, delay);
    });
}
```

Key Concept:

The Promise constructor takes a function with two parameters: `resolve` and `reject`. Call `resolve(value)` when the operation succeeds, or `reject(error)` when it fails.

Consuming Promises with .then() and .catch()

Now let's use this Promise-based function to fetch the same data we did in Part 1, but with much cleaner code:

```
getPromise("https://jsonplaceholder.typicode.com/users/1", 2000)
    .then(user => {
        console.log("User: ", user);
        return getPromise("https://jsonplaceholder.typicode.com/posts?userId=1", 2000);
    })
    .then(posts => {
        console.log("Post: ", posts[0]);
        return getPromise("https://jsonplaceholder.typicode.com/comments?postId=1", 2000);
    })
    .then(comments => {
        console.log("Comment: ", comments[0]);
    })
```

```
    .catch(err => {
        console.log(err);
    });
```

Output:

After 2 seconds:

User: {id: 1, name: "Leanne Graham", username: "Bret", ...}

After 4 seconds total:

Post: {userId: 1, id: 1, title: "sunt aut facere...", ...}

After 6 seconds total:

Comment: {postId: 1, id: 1, name: "id labore ex...", ...}

Advantages of Promises Over Callbacks

Compare this Promise-based code to the callback hell from Part 1. Notice how:

- **Flat Structure:** The code reads linearly from top to bottom, not nested deeply
- **Chaining:** Each `.then()` returns a new Promise, allowing operations to chain naturally
- **Centralized Error Handling:** A single `.catch()` at the end handles errors from any step
- **Readability:** The flow of data is clear and easy to follow

Async/Await: Syntactic Sugar for Promises

While Promises significantly improved asynchronous code, JavaScript introduced `async` and `await` keywords to make asynchronous code look and behave even more like synchronous code. This is often called "syntactic sugar" because it provides a sweeter, more pleasant syntax without changing the underlying functionality.

Powered by SPRK Technologies

The async Keyword

The `async` keyword is placed before a function declaration. It does two things:

- Makes the function always return a Promise
- Allows the use of `await` inside the function

The await Keyword

The `await` keyword can only be used inside `async` functions. It pauses the function execution until the Promise is resolved and returns the fulfilled value.

Refactoring with Async/Await

Let's rewrite our Promise chain using async/await:

```
async function loadData() {
    try {
        const user = await getPromise("https://jsonplaceholder.typicode.com/users/1", 2000);
        console.log("User: ", user);

        const posts = await getPromise("https://jsonplaceholder.typicode.com/posts?userId=1",
2000);
        console.log("Post: ", posts[0]);

        const comments = await getPromise("https://jsonplaceholder.typicode.com/comments?
postId=1", 2000);
        console.log("Comment: ", comments[0]);
    } catch (err) {
        console.error(err);
    }
}

loadData();
```

Output:

After 2 seconds:

Powered by SPRK Technologies

User: {id: 1, name: "Leanne Graham", username: "Bret", ...}

After 4 seconds total:

Post: {userId: 1, id: 1, title: "sunt aut facere...", ...}

After 6 seconds total:

Comment: {postId: 1, id: 1, name: "id labore ex...", ...}

This code is functionally identical to the Promise chain but reads like synchronous code. The `try/catch` block handles errors just like in synchronous code, making error handling intuitive and familiar.

The Fetch API: Modern HTTP Requests

The Fetch API is the modern replacement for XMLHttpRequest. It's Promise-based by default, cleaner to use, and has become the standard way to make HTTP requests in JavaScript.

Basic Fetch Syntax

The `fetch()` function takes a URL and returns a Promise that resolves to the Response object:

```
async function fetchJSON(url) {
    const response = await fetch(url); // fetch returns a Promise

    if (!response.ok) {
        throw new Error("Request failed");
    }

    return response.json(); // convert the response to JSON format
}
```

Key Concept:

Powered by SPRK Technologies

The `response.json()` method also returns a Promise, which is why we can await it. The `response.ok` property is a shorthand to check if the status code is in the 200-299 range.

Complete Example with Fetch

Let's implement the same user-posts-comments flow using the Fetch API:

```
async function fetchJSON(url) {
    const response = await fetch(url);

    if (!response.ok) {
        throw new Error("Request failed");
    }

    return response.json();
}

async function loadData() {
    try {
        const user = await fetchJSON("https://jsonplaceholder.typicode.com/users/1");
        console.log("User: ", user);

        const posts = await fetchJSON("https://jsonplaceholder.typicode.com/posts?userId=1");
        console.log("Post: ", posts[0]);

        const comments = await fetchJSON("https://jsonplaceholder.typicode.com/comments?
postId=1");
        console.log("Comment: ", comments[0]);
    } catch (err) {
        console.error(err);
    }
}

loadData();
```

Output:

User: {id: 1, name: "Leanne Graham", username: "Bret", ...}

Powered by SPRK Technologies

Post: {userId: 1, id: 1, title: "sunt aut facere...", ...}

Comment: {postId: 1, id: 1, name: "id labore ex...", ...}

The Evolution: A Side-by-Side Comparison

Let's review the same operation implemented with each approach we've learned:

1. Callbacks (Original Approach)

```
// Nested, difficult to read
delayedGet(url1, 2000, (err, data1) => {
    if (err) return console.error(err);
    console.log(data1);
    delayedGet(url2, 2000, (err, data2) => {
        if (err) return console.error(err);
        console.log(data2);
    });
});
```

2. Promises (Improved)

```
// Flat, chainable
getPromise(url1, 2000)
    .then(data1 => {
        console.log(data1);
        return getPromise(url2, 2000);
    })
    .then(data2 => console.log(data2))
    .catch(err => console.error(err));
```

3. Async/Await (Modern)

```
// Reads like synchronous code
async function getData() {
    try {
```

```
        const data1 = await getPromise(url1, 2000);
        console.log(data1);
        const data2 = await getPromise(url2, 2000);
        console.log(data2);
    } catch (err) {
        console.error(err);
    }
}
```

4. Fetch with Async/Await (Current Standard)

```
// Modern, clean, industry standard
async function getData() {
    try {
        const response1 = await fetch(url1);
        const data1 = await response1.json();
        console.log(data1);

        const response2 = await fetch(url2);
        const data2 = await response2.json();
        console.log(data2);
    } catch (err) {
        console.error(err);
    }
}
```

Best Practices and Key Takeaways

When to Use What

- **Async/Await with Fetch:** Use this for most modern applications. It's the cleanest and most readable approach
- **Promise Chains:** Useful when you need multiple operations to run in parallel using `Promise.all()`
- **Callbacks:** Still found in older codebases and some Node.js APIs, but avoid for new code
- **XMLHttpRequest:** Legacy approach, use only when supporting very old browsers

Powered by SPRK Technologies

Important Reminders

- Always use `try/catch` blocks with async/await for error handling
- Remember that `await` only works inside `async` functions
- Don't forget to check `response.ok` when using fetch—it doesn't automatically throw errors for HTTP error status codes
- `response.json()` is asynchronous and must be awaited

Performance Consideration

When operations don't depend on each other, consider running them in parallel:

```
// Sequential (slower - 6 seconds total)
const user = await fetch(url1);
const posts = await fetch(url2);
const comments = await fetch(url3);

// Parallel (faster - 2 seconds total)
const [user, posts, comments] = await Promise.all([
    fetch(url1),
    fetch(url2),
    fetch(url3)
]);
```

Output:

Sequential: Each request waits for the previous one to complete

Parallel: All requests start simultaneously and complete together

Conclusion

The evolution from callbacks to Promises to async/await represents JavaScript's continuous improvement in handling asynchronous operations. Each iteration built upon the previous one, addressing pain points and improving developer experience.

Modern JavaScript development primarily uses async/await with the Fetch API, combining the power of Promises with syntax that's easy to read and maintain. Understanding this progression not only helps you write better code but also prepares you to work with legacy codebases that may still use older patterns.

The journey from callback hell to clean, readable asynchronous code demonstrates how programming languages evolve to meet developers' needs, making complex operations simpler and more intuitive.